

RUN LENGTH ENCODING FOR FAST DATA COMPRESSION

A new routine, named RUNLEN, has been developed for fast Run Length Encoding. Data in conventional vector form can now be compressed into an alternate format, potentially saving much storage space. The routine provides a considerable performance improvement over conventional methods, achieving a best-case inner loop time of 0.333 microseconds per data point. In measurements against a VAX 11/780, the FPS-5205 is 18 times faster using the new routine.

RUN LENGTH ENCODING

In many image processing and computer graphics applications, large sections of data contain the same value. Usually this data is stored in 'image' form, where one word of memory is allocated for every data element, no matter what value it may contain. Run length encoding is a compression method that stores any contiguous area of data containing the same value in two words: one for the value, and one for the number of words containing that value. This method does not always result in data compression, as in the case where all data values are unique. When this occurs, the storage exactly doubles.

The most common example of an image that benefits from Run Length Encoding is typewritten text. Most of the page is white, with small solid black regions. In some cases, the compressed storage size is only one-tenth of the original image. The application which inspired the development of this routine was computer graphic animation. Large areas of the images contained regions of homogeneous color, such that storage requirements on 1600 bpi tape were significantly reduced for the large number of images required to build

animated sequences. To compress color images, the routine is called three times: once for each of the red, green, and blue image components.

VECTOR TO VECTOR COMPRESSION

The routine RUNLEN takes an input vector, and generates two new output vectors in compressed form. The final length of the compressed vector is also passed back in an SCM memory location.

"Data in conventional vector form can now be compressed into an alternate format, potentially saving much storage space."

The calling sequence is

```
CALL RUNLEN (INVEC,ININC,INNUM,
             CNTVEC,VALVEC,RUNNUM)
```

Where: INVEC = address of input data vector.

ININC = input data increment.

INNUM = length of input vector.

CNTVEC = address of output vector containing count integers.

VALVEC = address of output vector containing values.

RUNNUM = address of output length for CNTVEC and VALVEC.

This APAL routine takes as input a vector containing any input data set. The output is two vectors, and a scalar representing output vector lengths. The input data is packed into the output vector, and contiguous runs are stored where possible as the value and count, or number of input positions having that value. The parameter passing is the same as the standard FPS Math Library for the control processor, and may be called from APAL, MAXA1, VFC, CPFORTRAN, or host FORTRAN. The routine assumes all addresses are in the current page, and makes no modification to the page select registers.

The algorithm used in RUNLEN is shown below in VAX FORTRAN77 equivalent code.:

```
SUBROUTINE RUNLEN
(V,INCV,NVALUE,OUT,OUTLEN,N)

C  INPUTS:
C      V = VALUE ARRAY
C      INCV = INPUT INCREMENT IN THE
C      APAL VERSION, NOT USED HERE.
C      NVALUE = NUMBER OF INPUT
C      VALUES

C  OUTPUTS:
C      OUT = OUTPUT VALUES, PACKED
C      OUTLEN = RUN COUNT VALUES
C      N = NUMBER OF ELEMENTS IN
C      OUTPUT ARRAY

C      INTEGER N,I,J,NVALUE,INCV
C      REALV(1), OUT(1),OUTLEN(1)
C      N = 1
C      OUT(1)=V(1)
C      I= 1
C      J= 2
C      DO WHILE (J.LE.NVALUE)
C          I=1
C          DO WHILE (V(J).EQ.V(J-D .AND.
C              J.LE.NVALUE)
C              J= J+ 1
C              I= I+1
C          END DO
C          OUTLEN(N)= I
C          N= N+ 1
C          OUT(N)=V(J)
C          J= J+1
C      END DO
C      OUTLEN(N)= I
C      RETURN
C      END
```

PERFORMANCE

The routine RUNLEN has been tested on an FPS-5320 Control Processor, and compared to the above FORTRAN running on a VAX 11/780 with Floating Point Accelerator (FPA), under the VMS operating system. The APAL code executes over the input vector with a best case of two cycles per input point if all values are identical. Whenever the end of a contiguous segment is encountered, additional cycles are needed to store the result. The worst case timing is when no compression is possible. When every value is different than its neighbor, the inner loop becomes 18 cycles per four input points. The FPS-5320 has a cycle time of 0.157 microseconds, so the execution time ranges from 0.333 to 1.0 microseconds per input point.

Figure 1 shows the relative execution time for the VAX 11/780 and the FPS-5000, for various data compression ratios. The measurements were made on one million points of test data, organized as 10000 sets of 100-element vectors. The

left side of the graph represents run time in seconds, while the right side indicates performance improvement normalized to the VAX times. The AP routine is a consistent 18 times faster than the VAX, regardless of the compression ratio.

CONCLUSIONS

Normally these output values are further compressed by packing each count and value together into one small integer. This is done with library routines or with the GPIO I/O processor. The inverse operation, run length decoding, is performed using the standard Math Library Function VFILL. Since this routine is written as a general vector-to-vector operator, it may be integrated into other software systems easily, and makes no assumptions about the possible uses by the application program.

The RUNLEN routine is available free of charge with the ARRAY On-Line Library. For access to this library, contact Barbara Perz, Floating Point Systems, Inc., (800) 547-1445, extension 1445.



ian Curington is a Signal and Image Processing Specialist with Floating Point Systems U.K. Ltd., where his responsibilities include performance analysis, graphics, and image processing for northern Europe. His background includes both product marketing and performance analysis positions at Floating Point Systems

World Headquarters in Beaverton, Oregon. Prior to joining FPS in 1979, he worked in real-time control and micro-computer systems design. Mr. Curington holds a BS in mathematics from Lewis and Clark College. His professional interests include computer graphics, image synthesis, and motion dynamics. He is a member of ACM-SIGGRAPH, EUROGRAPHICS, and the Computer Arts Society.



Jan Boerhout is a Systems Engineer with Floating Point Systems in The Netherlands. His responsibilities include performance analysis and post-sales software support. Before coming to FPS, Mr. Boerhout was an Avionics Engineer and a Flight Simulation Software Specialist with Fokker Aircraft Industries in

The Netherlands. Mr. Boerhout holds degrees in electrical and electronics engineering, and his special interests include real-time continuous system simulation.

For additional information, contact Doug Atterbury, Software Product Support Manager, (800) 547-1445, extension 1680.

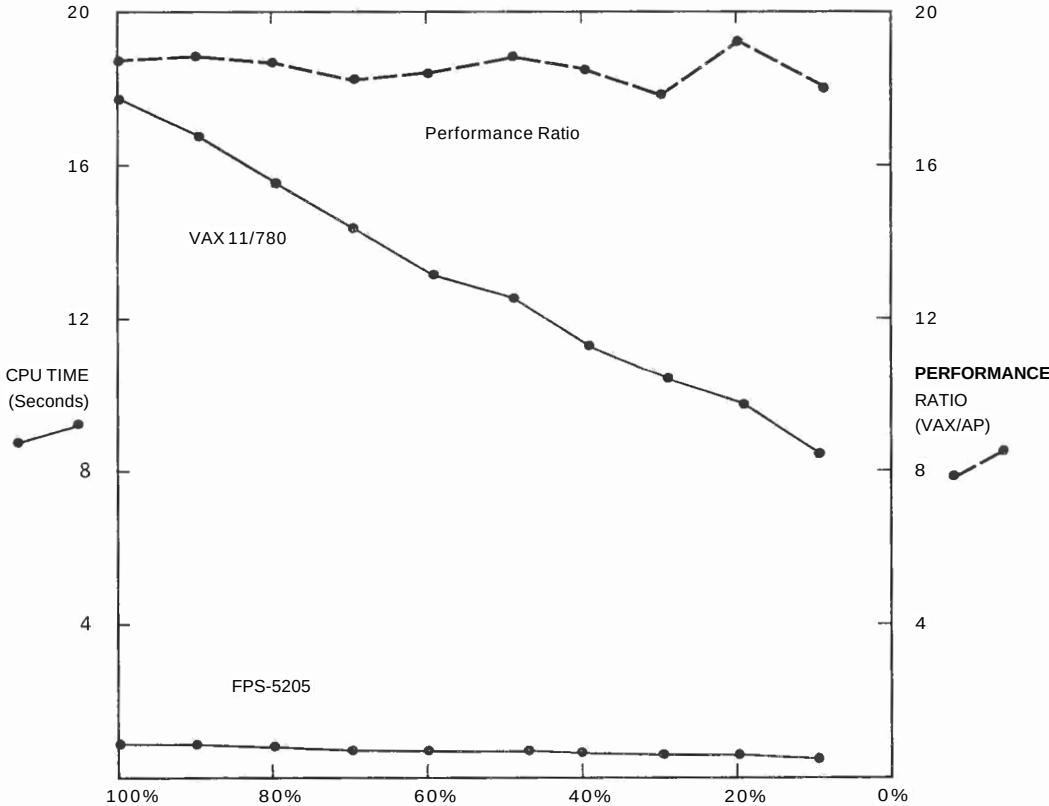


Figure 1: Size of Output as Percentage of Input Size